A GPU-Accelerated Implicit Material Point Method

Leo Jiang

15-618 Parallel Computer Architecture and Programming
Fall 2026

Project website: `https://leojiang12.github.io/gpu-mpm-project-website/`

1 Summary

We are implementing a GPU-Accelerated Implicit Material Point Method (MPM) solver and analyzing its performance characteristics to explore additional GPU optimizations.

2 Background

MPM is a numerical method for simulating deformable materials undergoing large deformations, including elastic solids, snow, and granular materials. It combines a Lagrangian particle representation with an Eulerian background grid, leveraging advantages of both representations. MPM tracks each particle's physical state throughout the simulation and updates it at each time step. MPM then leverages the Eulerian representation by transferring the particle states onto the grid to solve the equations of motion and then transferring the updated grid states back to the particles. The key advantage of MPM is that it evaluates material behavior at the particles while using a background grid to efficiently solve for the material's motion, avoiding the mesh distortion problems associated with Lagrangian mesh-based methods. MPM's particle-grid formulation is particularly useful for simulations involving large deformation and complex material motion.

MPM consists of the following stages: Particle-to-Grid (P2G) transfer, Grid Update, and Grid-to-Particle (G2P) transfer.

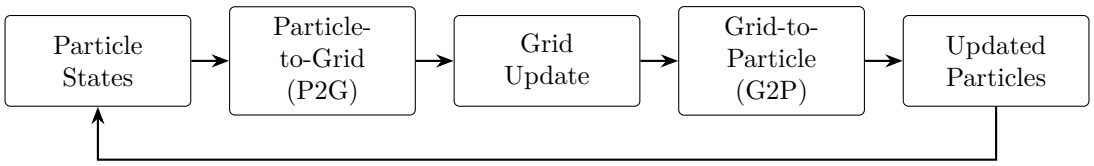

Figure 1: Overview of a Material Point Method simulation time step.

- P2G transfers particle mass, momentum, and forces to nearby grid nodes.
- Grid Update computes the next nodal state, explicitly or through an implicit optimization/linear solve.
- G2P interpolates updated grid quantities back to particles, updating their velocities, positions, and deformation gradients.

3 The Challenge

The first challenge of transferring information between particles and grid is irregular particle-grid memory accesses. If each GPU thread was assigned to a particle, threads within a warp may access widely separated grid locations because their particles are distributed throughout the domain.

The second challenge is write-conflicts during P2G. In a particle-parallel P2G implementation, many threads accumulate contributions into the same grid nodes, creating write conflicts. Atomic additions are a straightforward solution, but contention can serialize portions of the computation limiting speedup.

The third challenge is the grid update step for implicit time steps. Parallel sparse Hessian assembly itself is challenging. Each particle contributes to a small set of grid degrees of freedom, and these local contributions must be accumulated into a global sparse system. Overlapping particle stencils can produce write conflicts, just as they do in P2G. Matrix-free computation can serve as a potential optimization that trades matrix storage and assembly for additional operator computation.

4 Resources

I plan to develop my program on my laptop's RTX 5090 GPU and Intel(R) Core(TM) Ultra 9 285H (2.90 GHz). We plan to implement all of this from scratch using C++ with Eigen and CUDA. There is a paper with public source code on this [1] but we will not refer to the code. Additionally, we plan to use Nsight Systems and Nsight Compute to perform GPU profiling for our analyses.

5 Goals and Deliverables

5.1 Plan to Achieve

- CPU/GPU 2D Implicit MPM Solver
- CPU/GPU 3D Implicit MPM Solver
- CPU/GPU Speedup
- Problem-size Scaling
- Kernel Analysis of GPU Implementations
- GPU Bottlenecks

The core minimum results from this project are 2D CPU/GPU solvers with in-depth analysis on the challenges of GPU parallelization and exploration of optimizations for those challenges. As for visualization, we plan to include some short demo videos, at minimum for the 2D implementations. Since the focus of the project is geared toward the MPM solver, we plan to export frame

and geometry data to a third-party software like Blender (or just matplotlib for 2D) to render visualizations.

5.2 Hope to Achieve

- Polished 3D Visualizations
- Matrix-Free linear solver for Grid Update step
- Other materials (e.g. sand, plastic materials, etc.)

5.3 Fallback Scope

At minimum we change the trajectory of the project towards pure performance analysis on the workload of the 2D Implicit MPM CPU/GPU implementations.

6 Platform Choice

1. MPM has abundant particle-level parallelism, making GPU execution a natural choice.
2. Particle-grid transfers produce irregular memory access and atomic contention, making performance optimization nontrivial.
3. Implicit integration involves computationally intensive evaluations of elastic derivatives and repeated linear algebra, making GPU acceleration potentially beneficial.

7 Schedule

Week	Planned work
Nov. 9–15	2D CPU/GPU Solver
Nov. 16–22	3D CPU/GPU Solver
Nov. 23–29	**Milestone Nov. 23**; Performance Analysis and Targeted GPU Optimizations
Nov. 30–Dec. 6	Matrix-Free Grid Update + Other Materials (Time permitting)
Dec. 7–11	**Final report Dec. 10, poster Dec. 11**

References

[1] Ming Gao, Xinlei Wang, Kui Wu, Andre Pradhana, Eftychios Sifakis, Cem Yuksel, and Chenfanfu Jiang. GPU optimization of material point methods. *ACM Transactions on Graphics*, 37(6), Article 254, pages 1–12, December 2018. https://doi.org/10.1145/3272127.3275044